MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Călin Poiată, FAF-243

Report

Laboratory Work No. 3

Cryptography and Security

Topic: Polyalphabetic Ciphers

Implementation and Analysis of the Vigenère Cipher for the Romanian Alphabet

Checked by:
Maia Zaica, *university assistant*

Chișinău – 2026

CONTENTS

Introduction **3**

1 Short Theory **4**
1.1 Limitations of Monoalphabetic Systems 4
1.2 The Polyalphabetic Principle 4
1.3 Mathematical Model of the Vigenère Cipher 4
1.4 The 31-Letter Romanian Alphabet 5
1.5 Tabula Recta for the Romanian Alphabet 5
1.6 Cryptanalytic Vulnerabilities of Periodic Ciphers 6

2 Implementation Architecture **7**
2.1 Program Architecture and Components 7
2.2 Alphabet Encoding and Diacritic Normalization 7
2.3 Input Sanitization and Range Validation 7
2.4 Key Length Enforcement 8
2.5 Modular Calculation Engine and Table Display 8
2.6 Interactive User Interface 8

3 Python Source Code Listings **9**
3.1 Alphabet Mapping and Normalization 9
3.2 Input Validation Routines 9
3.3 Encryption, Decryption, and Tabular Output 11

4 Experimental Results and Verification **12**
4.1 Test Case 1: Standard Academic Text 12
4.2 Test Case 2: Diacritic-Dense Romanian Message 13
4.3 Test Case 3: Error Handling and Constraint Enforcement 13
4.3.1 Enforcing Key Length Constraint $|K| \geq 7$ 13
4.3.2 Enforcing Valid Character Range 14

5 Cryptographic Discussion and Security Evaluation **15**
5.1 Key Space Analysis 15
5.2 Comparison: Periodic Vigenère vs. One-Time Pad 15

6 Conclusions **17**

INTRODUCTION

The primary objective of classical cryptography has historically been the design of ciphers capable of concealing statistical structures inherent to natural languages. In Laboratory Work No. 2, the vulnerabilities of monoalphabetic substitution were directly experienced: because a monoalphabetic cipher establishes a fixed bijection between the plaintext alphabet and the ciphertext alphabet, it completely preserves unigram, digram, and trigram frequencies. Consequently, frequency analysis, repeated pattern inspection, and vowel-consonant clustering allow an intercept to be fully deciphered without prior knowledge of the secret permutation.

Polyalphabetic ciphers represent a pivotal historical and conceptual evolution designed to flatten this non-uniform frequency distribution. By substituting each letter using a variable permutation determined by its position and a repeating key, polyalphabetic algorithms obscure the direct relationship between message statistics and ciphertext symbols. The most famous and historically significant polyalphabetic cipher is the *Vigenère cipher*, famously hailed for centuries as *le chiffre indéchiffrable* (the unbreakable cipher).

In this laboratory work, I implemented and evaluated **Task 3.2**, assigned to even-numbered students in the academic register. The core mandate comprises:

1. Implementing the Vigenère encryption and decryption algorithm in Python adapted specifically for the **31-letter Romanian alphabet** ($A, \breve{A}, \hat{A}, \ldots, Z$), whose characters are numerically codified in the ring $\mathbb{Z}_{31} = \{0, 1, \ldots, 30\}$.

2. Enforcing rigorous character validation: the input domain is strictly restricted to valid letters (A–Z, a–z and Romanian diacritics ă, â, î, ș, ț). If an unauthorized symbol (digit, punctuation, special character) is entered, the system rejects the input and informs the user of the permissible character range.

3. Enforcing a minimum key length constraint: $|K| \geq 7$, ensuring that the polyalphabetic key space is non-trivial.

4. Implementing message preprocessing according to specifications: whitespace elimination followed by conversion to uppercase.

5. Supporting bi-directional operation (encryption and decryption) based on the modular arithmetic model:
$$c_i = (m_i + k_i) \pmod{31}, \qquad m_i = (c_i - k_i) \pmod{31}. \tag{1}$$

6. Providing an educational tabular trace demonstrating the numeric modular transformations step-by-step.

1 SHORT THEORY

1.1 Limitations of Monoalphabetic Systems

A monoalphabetic cipher operates under a fixed permutation $\pi : \Sigma \to \Sigma$ such that ciphertext letter $c_i = \pi(m_i)$. If a language has probability distribution $P(X = \alpha)$ for letters $\alpha \in \Sigma$, then the ciphertext letter $\beta = \pi(\alpha)$ appears with probability:

$$P(Y = \beta) = P(X = \pi^{-1}(\beta)) = P(X = \alpha). \tag{2}$$

Because the entropy and relative frequencies are invariant under a static bijection, statistical attacks succeed quickly even when the theoretical key space $|\Sigma|!$ is astronomically large (e.g., $26! \approx 4.03 \times 10^{26}$ or $31! \approx 8.22 \times 10^{33}$).

1.2 The Polyalphabetic Principle

To eliminate monoalphabetic frequency leakage, polyalphabetic ciphers use multiple translation alphabets $\pi_0, \pi_1, \ldots, \pi_{L-1}$ cyclically:

$$c_i = \pi_{i \pmod L}(m_i). \tag{3}$$

If letter 'E' appears at positions i_1 and i_2 where $i_1 \not\equiv i_2 \pmod L$, it is encrypted under different permutations and yields different ciphertext characters. Conversely, identical ciphertext letters may originate from distinct plaintext characters. This combines high-frequency and low-frequency distributions, flattening the overall histogram toward a uniform distribution.

The foundational concept was formulated by Leon Battista Alberti in 1467 using concentric cipher disks. In 1518, Johannes Trithemius introduced the *Tabula Recta* (a square matrix of shifted alphabets). Giovan Battista Bellaso in 1553 introduced the keyword-driven polyalphabetic mechanism. Blaise de Vigenère subsequently published an autokey variation in 1586, though historical convention later associated Bellaso's keyword cipher with Vigenère's name.

1.3 Mathematical Model of the Vigenère Cipher

Let Σ be an alphabet of cardinality $N = |\Sigma|$, and let each character be associated with a unique index in the residue ring $\mathbb{Z}_N$:

$$f : \Sigma \longleftrightarrow \mathbb{Z}_N = \{0, 1, \ldots, N-1\}. \tag{4}$$

Let the plaintext message be represented by indices $\mathbf{m} = (m_0, m_1, \ldots, m_{n-1})$ and the key by indices $\mathbf{k} = (k_0, k_1, \ldots, k_{L-1})$ of length $L \geq 7$. The keystream is extended periodically over the length of the message:

$$\tilde{k}_i = k_{i \pmod L}, \quad \text{for } 0 \leq i < n. \tag{5}$$

The encryption transformation $E_k : \mathbb{Z}_N \to \mathbb{Z}_N$ is defined by modular addition:

$$c_i = (m_i + \tilde{k}_i) \pmod N. \tag{6}$$

The decryption transformation $D_k : \mathbb{Z}_N \to \mathbb{Z}_N$ is the additive inverse under modulo N:

$$m_i = (c_i - \tilde{k}_i) \pmod N. \tag{7}$$

Equations (6) and (7) satisfy the correctness guarantee $D_k(E_k(m_i)) = m_i$ because:

$$(c_i - \tilde{k}_i) \pmod N = ((m_i + \tilde{k}_i) - \tilde{k}_i) \pmod N = m_i \pmod N = m_i. \tag{8}$$

1.4 The 31-Letter Romanian Alphabet

Task 3.2 specifies adapting the Vigenère algorithm to the Romanian alphabet containing 31 letters. The official Romanian alphabet extends the 26 Latin letters with 5 specific diacritical letters: Ă, Â, Î, Ș, and Ț.

In standard Romanian alphabetical order, these 31 characters are indexed from 0 to 30, as outlined in Table 1.

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	–
Index	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–

Table 1: Numerical codification of the 31-letter Romanian alphabet ($\mathbb{Z}_{31}$).

An important mathematical property of this adaptation is that the alphabet size $N = 31$ is a **prime number**. In abstract algebra, $\mathbb{Z}_{31} = \mathbb{Z}/31\mathbb{Z}$ forms a finite field $\mathbb{F}_{31}$. Consequently:

- Every non-zero shift $k_i \in \{1, 2, \ldots, 30\}$ is coprime to 31, meaning $\gcd(k_i, 31) = 1$.
- Under repeated addition of any non-zero constant k, the orbit generates the entire group $\mathbb{Z}_{31}$ with maximal period 31, eliminating non-trivial sub-period degeneracies that can occur when N is composite.

1.5 Tabula Recta for the Romanian Alphabet

The classical geometric representation of the cipher is the *Tabula Recta*, a square matrix of dimension 31×31 where each subsequent row is cyclically shifted by one position to the left. The row corresponding to key letter k and the column corresponding to plaintext letter m intersect at ciphertext letter $c = (m + k) \pmod{31}$.

	A	**Ă**	**Â**	**B**	**C**	**D**	**E**	…	**Ș**	**T**	**Ț**	**U**	…	**Y**	**Z**
A (0)	A	Ă	Â	B	C	D	E	…	Ș	T	Ț	U	…	Y	Z
Ă (1)	Ă	Â	B	C	D	E	F	…	T	Ț	U	V	…	Z	A
Â (2)	Â	B	C	D	E	F	G	…	Ț	U	V	W	…	A	Ă
B (3)	B	C	D	E	F	G	H	…	U	V	W	X	…	Ă	Â
C (4)	C	D	E	F	G	H	I	…	V	W	X	Y	…	Â	B
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋱	⋮	⋮	⋮	⋮	⋱	⋮	⋮
Z (30)	Z	A	Ă	Â	B	C	D	…	S	Ș	T	Ț	…	W	X

Table 2: Excerpt of the 31×31 Tabula Recta for the Romanian alphabet.

While the physical Tabula Recta was indispensable for manual cryptographers in the sixteenth to nineteenth centuries, computerized execution performs the direct modular addition and subtraction in $O(1)$ time per character.

1.6 Cryptanalytic Vulnerabilities of Periodic Ciphers

Despite its historical reputation, the periodic Vigenère cipher is not unconditionally secure. When the key length L is fixed and repeats periodically, the cryptogram can be factored into L independent monoalphabetic sub-ciphers:

$$\mathbf{C}^{(j)} = (c_j, c_{j+L}, c_{j+2L}, \dots), \quad \text{for } 0 \le j < L. \tag{9}$$

Each sub-cipher $\mathbf{C}^{(j)}$ is encrypted with a single Caesar shift k_j. Once L is determined, standard monoalphabetic frequency analysis or chi-squared goodness-of-fit tests recover each k_j independently.

The key length L can be deduced using two primary methods:

1. **Kasiski Examination (1863):** Identical plaintext n-grams encrypted under the same key position yield identical ciphertext n-grams. The distance d between repeated ciphertext strings will be a multiple of the key length L. By finding the greatest common divisor (gcd) of observed distances, candidate values of L are revealed.

2. **Friedman Test / Index of Coincidence (1925):** The Index of Coincidence (IC) measures the probability that two randomly selected letters from a text are identical:

$$IC = \frac{\sum_{i=0}^{N-1} f_i(f_i - 1)}{n(n-1)}. \tag{10}$$

   For natural Romanian language, $IC_{\text{lang}} \approx 0.075$, while for a completely random text over 31 letters, $IC_{\text{rand}} = \frac{1}{31} \approx 0.0323$. As the key length L increases, the overall ciphertext IC drops toward 0.0323. By testing trial periods k, the correct period L exhibits an average coset IC that surges back up to ≈ 0.075.

Enforcing a minimum key length $|K| \ge 7$, as required by Task 3.2, prevents trivial manual frequency cracking on short messages, because at least 7 cosets are interleaved.

2 IMPLEMENTATION ARCHITECTURE

2.1 Program Architecture and Components

The software is implemented in Python (`vigenere.py`) to fulfill all specifications of Task 3.2. The application architecture is structured into four functional modules:

1. **Alphabet Representation and Encoding:** Establishes the 31-character Romanian alphabet mapping to $\mathbb{Z}_{31} = \{0, 1, \ldots, 30\}$ using bidirectional hash tables for $O(1)$ lookups, along with diacritic normalization for character variants.
2. **Sanitization and Validation Pipeline:** Implements the required input transformations (whitespace elimination followed by uppercase conversion) and enforces alphabet range validation and key length constraints ($|K| \geq 7$).
3. **Modular Cryptographic Engine:** Executes the algebraic encryption and decryption transformations ($c_i = (m_i + k_i) \pmod{31}$ and $m_i = (c_i - k_i) \pmod{31}$) and formats the step-by-step arithmetic calculation table.
4. **Interactive User Interface:** Provides a console menu allowing the operator to select encryption or decryption, enter messages and keys, and view the resulting outputs.

2.2 Alphabet Encoding and Diacritic Normalization

The Romanian alphabet contains characters with diacritical marks. In Unicode, Romanian text may be represented using modern comma-below characters (preferred by ISO/IEC and the Romanian Academy) or legacy cedilla characters:

- Standard comma-below: Ș (`U+0218`), ș (`U+0219`), Ț (`U+021A`), ț (`U+021B`).
- Legacy cedilla: Ş (`U+015E`), ş (`U+015F`), Ţ (`U+0162`), ţ (`U+0163`).

To guarantee seamless user experience across different keyboard layouts and terminal encodings, the preprocessing module normalizes legacy cedilla glyphs to standard comma-below representations before validation.

2.3 Input Sanitization and Range Validation

Task 3.2 specifies:

> *„Valorile caracterelor textului sunt cuprinse între 'A' și 'Z', 'a' și 'z' și nu sunt permise alte valori. În cazul în care utilizatorul introduce alte valori – i se va sugera diapazonul corect al caracterelor. În mesaj mai întâi trebuie eliminate spațiile, apoi toate literele se vor transforma în majuscule."*

The processing pipeline executes the following stages:

1. **Whitespace Elimination:** All spaces, tabs, and newline characters are stripped using Python's `"".join(text.split())`.
2. **Case Transformation:** All remaining characters are converted to uppercase via `.upper()`. Python's Unicode library correctly handles Romanian diacritic case conversion (ă → Ă, ș → Ș, etc.).
3. **Alphabet Range Verification:** Every character is evaluated against the 31-letter set. If any non-alphabetical characters (e.g., digits 0–9, punctuation marks, or mathematical symbols) are found, the system isolates the illegal characters and displays an explicit diagnostic message instructing the user that only letters from the Romanian alphabet are accepted.

2.4 Key Length Enforcement

The specification mandates: *„Lungimea cheii nu trebuie să fie mai mică de 7."* The function `validate_key` validates that:

$$|K_{\text{cleaned}}| \geq 7. \tag{11}$$

If the cleaned key contains fewer than 7 characters, an informative error message is returned displaying the current length and prompting the user to supply a valid key.

2.5 Modular Calculation Engine and Table Display

The mathematical operations are implemented directly:

- **Encryption:** `c_idx = (CHAR_TO_IDX[ch] + CHAR_TO_IDX[k_ch]) % 31`
- **Decryption:** `m_idx = (CHAR_TO_IDX[ch] - CHAR_TO_IDX[k_ch]) % 31`

To provide complete visibility into the cipher's mechanics, the function `display_calculation_table` outputs a formatted breakdown identical to the worked example in the laboratory handout:

- Row 1: Source text characters (M or C).
- Row 2: Keystream characters (K).
- Row 3: Numeric indices of the source text ($0 \leq m_i \leq 30$).
- Row 4: Numeric indices of the keystream ($0 \leq k_i \leq 30$).
- Row 5: Arithmetic modulo result ($(m_i + k_i) \bmod 31$ or $(c_i - k_i) \bmod 31$).
- Row 6: Resulting output characters.

2.6 Interactive User Interface

In accordance with the specification:

> *„Utilizatorul va putea alege operația – criptare sau decriptare, va putea introduce cheia, mesajul sau criptograma și va obține criptograma sau mesajul decriptat."*

The application exposes a console-driven workflow providing:

- A main menu prompting the user to choose between encryption, decryption, alphabet inspection, test suite execution, or program termination.
- Continuous input loops that re-prompt upon encountering invalid inputs (undersized keys or illegal characters) while printing explicit diagnostic guidance.
- Clear presentation of the processed plaintext, active key, resulting cryptogram, and formatted modular calculation table.

3 PYTHON SOURCE CODE LISTINGS

The complete implementation is self-contained in `vigenere.py`. The core functional components are presented below.

3.1 Alphabet Mapping and Normalization

Listing 1 establishes the 31-letter Romanian alphabet, bidirectional hash maps, and the Unicode normalization translation table.

```python
# Romanian alphabet with 31 letters in standard alphabetical order
ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSȘTȚUVWXYZ"
N = len(ALPHABET)
CHAR_TO_IDX = {ch: i for i, ch in enumerate(ALPHABET)}
IDX_TO_CHAR = {i: ch for i, ch in enumerate(ALPHABET)}

# Maps legacy Romanian cedilla characters to standard comma-below variants
CEDILLA_MAP = str.maketrans({"Ş": "Ș", "ş": "ș", "Ţ": "Ț", "ţ": "ț"})

def clean_text(text: str) -> str:
    text = text.translate(CEDILLA_MAP)
    # Remove all whitespace before converting to uppercase
    text = "".join(text.split())
    return text.upper()

def find_invalid_chars(text: str) -> list[str]:
    # Returns characters that do not belong to the 31-letter Romanian alphabet
    return sorted(list({ch for ch in text if ch not in CHAR_TO_IDX}))
```

Listing 1: Alphabet definition and character normalization

3.2 Input Validation Routines

Listing 2 implements strict validation for both the secret key and the user text.

```python
def validate_key(raw_key: str) -> tuple[str | None, str | None]:
    cleaned = clean_text(raw_key)
    if len(cleaned) < 7:
        return None, f"Eroare: Lungimea cheii trebuie să fie de cel puțin 7 caractere (lungime actuală: {len(cleaned)})."
    invalid = find_invalid_chars(cleaned)
    if invalid:
        inv_str = ", ".join(f"'{c}'" for c in invalid)
        hint = (
            f"Eroare: Cheia conține caractere nepermise: {inv_str}.\n"
            f"Diapazonul permis: literele alfabetului român (A-Z, Ă, Â, Î, Ș, Ț). "
            f"Cifrele și semnele de punctuație nu sunt permise."
        )
        return None, hint
    return cleaned, None

def validate_message(raw_msg: str) -> tuple[str | None, str | None]:
    cleaned = clean_text(raw_msg)
    if not cleaned:
        return None, "Eroare: Mesajul introdus este gol."
    invalid = find_invalid_chars(cleaned)
    if invalid:
        inv_str = ", ".join(f"'{c}'" for c in invalid)
        hint = (
            f"Eroare: Textul conține caractere nepermise: {inv_str}.\n"
            f"Diapazonul permis: literele alfabetului român (A-Z, Ă, Â, Î, Ș, Ț). "
            f"Cifrele și semnele de punctuație nu sunt permise."
```

```
        )
        return None, hint
    return cleaned, None
```

Listing 2: Validation functions for key and message

3.3 Encryption, Decryption, and Tabular Output

Listing 3 contains the core mathematical transformations and the educational output table generator.

```python
def encrypt(plaintext: str, key: str) -> str:
    res = []
    key_len = len(key)
    for i, ch in enumerate(plaintext):
        k_ch = key[i % key_len]
        # Mathematical model: c_i = (m_i + k_i) mod 31
        c_idx = (CHAR_TO_IDX[ch] + CHAR_TO_IDX[k_ch]) % N
        res.append(IDX_TO_CHAR[c_idx])
    return "".join(res)

def decrypt(ciphertext: str, key: str) -> str:
    res = []
    key_len = len(key)
    for i, ch in enumerate(ciphertext):
        k_ch = key[i % key_len]
        # Mathematical model: m_i = (c_i - k_i) mod 31
        m_idx = (CHAR_TO_IDX[ch] - CHAR_TO_IDX[k_ch]) % N
        res.append(IDX_TO_CHAR[m_idx])
    return "".join(res)

def display_calculation_table(text: str, key: str, is_encrypt: bool = True) -> None:
    op_symbol = "+" if is_encrypt else "-"
    key_len = len(key)
    repeated_key = "".join(key[i % key_len] for i in range(len(text)))
    text_nums = [CHAR_TO_IDX[c] for c in text]
    key_nums = [CHAR_TO_IDX[c] for c in repeated_key]

    if is_encrypt:
        res_nums = [(t + k) % N for t, k in zip(text_nums, key_nums)]
    else:
        res_nums = [(t - k) % N for t, k in zip(text_nums, key_nums)]
    res_chars = [IDX_TO_CHAR[n] for n in res_nums]

    col_width = 4
    header1 = "Text (M):" if is_encrypt else "Criptogramă (C):"
    header2 = "Cheie (K):"
    header3 = f"M {op_symbol} K (mod 31):" if is_encrypt else f"C {op_symbol} K (mod 31):"
    header4 = "Rezultat:"

    chunk_size = 20
    for start in range(0, len(text), chunk_size):
        end = min(start + chunk_size, len(text))
        print("\n" + "-" * 75)
        print(f"{header1:<22} " + " ".join(f"{text[i]:>{col_width}}" for i in range(start, end)))
        print(f"{header2:<22} " + " ".join(f"{repeated_key[i]:>{col_width}}" for i in range(start, end)))
        print(f"{'Valori text:':<22} " + " ".join(f"{text_nums[i]:>{col_width}}" for i in range(start, end)))
        print(f"{'Valori cheie:':<22} " + " ".join(f"{key_nums[i]:>{col_width}}" for i in range(start, end)))
        print(f"{header3:<22} " + " ".join(f"{res_nums[i]:>{col_width}}" for i in range(start, end)))
        print(f"{header4:<22} " + " ".join(f"{res_chars[i]:>{col_width}}" for i in range(start, end)))
    print("-" * 75)
```

Listing 3: Cryptographic engine and calculation table generator

4 EXPERIMENTAL RESULTS AND VERIFICATION

To evaluate the functionality, accuracy, and fault tolerance of the software, multiple experiments were conducted covering standard text, diacritic-rich sentences, and invalid inputs.

4.1 Test Case 1: Standard Academic Text

Input message: `"UNIVERSITATEA TEHNICA A MOLDOVEI"`
Secret key: `"CRIPTOGRAFIE"` (length $|K| = 12 \geq 7$)

1. Preprocessing Stage:

- Spaces stripped: `"UNIVERSITATEATEHNICAAMOLDOVEI"` (length = 29 characters).
- Characters verified: all belong to Σ_{ro}.
- Key verified: length $12 \geq 7$, characters all belong to Σ_{ro}.

2. Encryption Step-by-Step Table:

Table 3 displays the numerical modular addition $(M + K)$ mod 31 for the complete message.

Text M	U	N	I	V	E	R	S	I	T	A	T	E	A	T	E
Key K	C	R	I	P	T	O	G	R	A	F	I	E	C	R	I
Val. M	25	16	10	26	6	20	21	10	23	0	23	6	0	23	6
Val. K	4	20	10	18	23	17	8	20	0	7	10	6	4	20	10
Sum mod31	29	5	20	13	29	6	29	30	23	7	2	12	4	12	16
Cipher C	Y	D	R	K	Y	E	Y	Z	T	F	Â	J	C	J	N
Text M	H	N	I	C	A	A	M	O	L	D	O	V	E	I	–
Key K	P	T	O	G	R	A	F	I	E	C	R	I	P	T	–
Val. M	9	16	10	4	0	0	15	17	14	5	17	26	6	10	–
Val. K	18	23	17	8	20	0	7	10	6	4	20	10	18	23	–
Sum mod31	27	8	27	12	20	0	22	27	20	9	6	5	24	2	–
Cipher C	W	G	W	J	R	A	Ș	W	R	H	E	D	Ț	Â	–

Table 3: Modular calculation table for Test Case 1 under encryption ($N = 31$).

Resulting Cryptogram:

```
YDRKYEYZTFÂJCJNWGWJRAȘWRHEDȚÂ
```

3. Decryption Verification:

Feeding the cryptogram `"YDRKYEYZTFÂJCJNWGWJRAȘWRHEDȚÂ"` and the key `"CRIPTOGRAFIE"` into the decryption module produces:

```
UNIVERSITATEATEHNICAAMOLDOVEI
```

The output matches the preprocessed plaintext character-for-character, validating mathematical inversion.

4.2 Test Case 2: Diacritic-Dense Romanian Message

To thoroughly verify diacritical mapping across the entire residue set $\mathbb{Z}_{31}$, a sentence featuring multiple Romanian diacritical letters was tested.

Input message: `"Învățământul este cheia succesului"`

Secret key: `"SIGURANȚĂ"` (length $|K| = 9 \geq 7$)

1. Preprocessed Plaintext:

```
ÎNVĂȚĂMÂNTULESTECHEIASUCCESULUI
```

Length: 31 characters.

2. Modular Encryption Calculation:

Table 4 illustrates the resulting arithmetic operations mod 31.

Text M	Î	N	V	Ă	Ț	Ă	M	Â	N	T	U	L	E	S	T	E
Key K	S	I	G	U	R	A	N	Ț	Ă	S	I	G	U	R	A	N
Val. M	11	16	26	1	24	1	15	2	16	23	25	14	6	21	23	6
Val. K	21	10	8	25	20	0	16	24	1	21	10	8	25	20	0	16
Sum mod31	1	26	3	26	13	1	0	26	17	13	4	22	0	10	23	22
Cipher C	Ă	V	B	V	K	Ă	A	V	O	K	C	Ș	A	I	T	Ș
Text M	C	H	E	I	A	S	U	C	C	E	S	U	L	U	I	–
Key K	Ț	Ă	S	I	G	U	R	A	N	Ț	Ă	S	I	G	U	–
Val. M	4	9	6	10	0	21	25	4	4	6	21	25	14	25	10	–
Val. K	24	1	21	10	8	25	20	0	16	24	1	21	10	8	25	–
Sum mod31	28	10	27	20	8	15	14	4	20	30	22	15	24	2	4	–
Cipher C	X	I	W	R	G	M	L	C	R	Z	Ș	M	Ț	Â	C	–

Table 4: Modular calculation table for Test Case 2 featuring extensive diacritics.

Ciphertext Produced:

```
ĂVBVKĂAVOKCȘAITȘXIWRGMLCRZȘMȚÂC
```

Decryption Verification:

Executing decryption with key `"SIGURANȚĂ"` restores the original message:

```
ÎNVĂȚĂMÂNTULESTECHEIASUCCESULUI
```

4.3 Test Case 3: Error Handling and Constraint Enforcement

The program was subjected to edge-case and boundary inputs to verify strict adherence to the problem statement.

4.3.1 Enforcing Key Length Constraint $|K| \geq 7$

When an undersized key is provided:

```
Introduceți cheia (minim 7 caractere): SECRET

Eroare: Lungimea cheii trebuie să fie de cel puțin 7 caractere (lungime actuală: 6).
Introduceți cheia (minim 7 caractere):
```

The program halts execution of the cipher operation, displays the diagnostic error, and prompts for a compliant key.

4.3.2 Enforcing Valid Character Range

When a user attempts to encrypt text containing digits, punctuation, or symbols:

```
Introduceți mesajul clar: Criptografia in 2026 este esentiala!

Eroare: Textul conține caractere nepermise: '!', '0', '2', '6'.
Diapazonul permis: literele alfabetului român (A-Z, Ă, Â, Î, Ș, Ț). Cifrele și semnele de punctuație nu sunt permise.
Introduceți mesajul clar:
```

The system isolates the exact offending glyphs (`'!', '0', '2', '6'`) and reminds the user of the permissible alphabet domain.

5 CRYPTOGRAPHIC DISCUSSION AND SECURITY EVALUATION

5.1 Key Space Analysis

In a brute-force exhaustive search scenario where the attacker attempts every possible key combination:

$$|\mathcal{K}_L| = N^L = 31^L. \tag{12}$$

Table 5 shows the rapid exponential growth of the key space as a function of key length L for the 31-letter Romanian alphabet.

Key Length L	**Total Combinations** (31^L)	**Effective Entropy** ($\approx \log_2(31^L)$)
1 (Caesar)	31	4.95 bits
3	29,791	14.86 bits
5	28,629,151	24.77 bits
6	887,503,681	29.73 bits
7 (Laboratory Minimum)	**27,512,614,111**	**34.68 bits**
10	819,628,286,980,801	49.54 bits
12	788,294,220,111,940,001	59.45 bits
16	727,880,181,961,846,900,000,000	79.27 bits

Table 5: Key space size and entropy for various key lengths over $\mathbb{Z}_{31}$.

At the laboratory minimum key length $L = 7$, an exhaustive search requires testing over 2.75×10^{10} keys, rendering purely manual attacks infeasible. For keys of length $L \geq 12$, the search space exceeds 7.88×10^{17} candidate keys (≈ 60 bits of entropy).

5.2 Comparison: Periodic Vigenère vs. One-Time Pad

The fundamental flaw of the standard Vigenère cipher is keystream periodicity:

$$\tilde{k}_{i+L} = \tilde{k}_i. \tag{13}$$

Because the keystream repeats every L characters, an adversary who gathers a ciphertext of length $M \gg L$ can distribute ciphertext symbols into L bins. Each bin constitutes a simple monoalphabetic substitution whose letter frequency distribution directly reflects the plaintext language distribution shifted by k_j.

In 1917, Gilbert Vernam proposed eliminating this periodic recurrence by generating a truly random keystream equal in length to the message ($L = |M|$) and using it exactly once. In 1949, Claude Shannon published his landmark paper *Communication Theory of Secrecy Systems*, mathematically proving that if:

1. The key is uniformly distributed: $P(K = k) = \frac{1}{|\mathcal{K}|}$,
2. The key is at least as long as the message: $|K| \geq |M|$,
3. The key is used exactly once and never reused,

then the cipher achieves **perfect secrecy** (information-theoretic security):

$$P(M = m \mid C = c) = P(M = m), \quad \forall m \in \mathcal{M}, c \in \mathcal{C}. \tag{14}$$

In such a system (the One-Time Pad), intercepting the ciphertext provides zero Shannon mutual information regarding the underlying plaintext ($I(M;C) = 0$). Thus, the Vigenère cipher represents both the conceptual ancestor of modern stream ciphers and the stepping stone toward information-theoretically secure encryption.

6 CONCLUSIONS

In this laboratory work, I successfully implemented and analyzed the Vigenère polyalphabetic cipher adapted for the 31-letter Romanian alphabet, fulfilling all requirements of Task 3.2.

The key accomplishments and findings include:

1. **Mathematical Rigor in $\mathbb{Z}_{31}$:** The standard 26-letter Caesar/Vigenère formulations were successfully generalized to the ring of integers modulo 31. Because 31 is prime, the algebraic structure $(\mathbb{Z}_{31}, +, \cdot)$ forms a finite field $\mathbb{F}_{31}$, guaranteeing that every non-zero key element acts as a generator of maximal additive order 31.

2. **Robust Character and Diacritic Handling:** The implementation accommodates modern Romanian comma-below diacritics (ș, ț) as well as legacy cedilla forms (ş, ţ), standardizing them cleanly.

3. **Strict Input Validation:** The program enforces the problem constraints by eliminating whitespace, transforming characters to uppercase, validating characters against the permissible Romanian alphabet domain, and providing diagnostic feedback when unauthorized characters are detected.

4. **Key Length Enforcement:** The requirement $|K| \geq 7$ is strictly enforced, ensuring an initial brute-force search space exceeding 2.75×10^{10} permutations.

5. **Bidirectional Correctness:** Experimental verification confirmed that $D_K(E_K(M)) = M$ across standard and diacritic-rich Romanian test texts. Detailed tabular execution traces confirmed that each numeric modular calculation matches theoretical expectations.

Through this implementation, the transition from monoalphabetic to polyalphabetic encryption was thoroughly demonstrated, illustrating both the significant security gains achieved by flattening frequency distributions and the inherent structural limitations of periodic keystreams.